

RFID Attendance System

With face verification and cloud logging

lot final project

Group2

Date: 17/07/2026

DOCUMENTATION

Overview

This project is an attendance tracking system that is built around ESP32, RFID, ESP32 Cam, and more. Basically, the system is a problem solver that help you verify whether the participant or students have come to class or not, and to avoid being lied on like how they can just sign attendance for each other by copying their friends signature and let one student sign it, we use ID card to verify if the person is here and also verify if the person holding the card is truly the owner.

After the system starts, the individual person can tap their card lightly against the RFID card scanner and then compose themselves for a face check. Failure attempt will result in a 3 second beep. If it is a success, it will only leave a 0.3 second beep.

Key features

- RFID card scan with live face verification through web
- logs can be access by multi-devices through fire store
- physical I2C LCD for real time status feedback
- will display the information = name + id + is the id valid-unknown-recognized
- multiple angles face reference photos
- RGB color for webcam
- will display right card right person, right card wrong person, right card unknown person, wrong card + nothing will display
- long beep 3s = wrong
- short beep 0.3s = correct

Components

1. **RC522 RFID reader:** This is the sensor that reads the student ID cards. When a card is tapped near it, it reads the card's unique UID (a fixed serial number burned into the card) over SPI. Your code looks up that UID in `STUDENT_DB` to figure out whose card it is. It doesn't read a name or student ID directly off the card, and the UID is just a fingerprint that your code maps to a person.
2. **ESP 32:** The microcontroller running the `main.py` (via Thonny/MicroPython). It reads the RFID reader, drives the LCD and buzzer, connects to WiFi, talks to Flask server, and writes to both the SD card and Firestore.
3. **ESP 32 Cam:** Second ESP32 board with a built in camera module.
4. **Buzzer:** Simple piezo buzzer wired to GPIO4 on the ESP32.
5. **I2C LCD:** Shows real time status text.
6. **SD card:** Physical microSD card, read through an SPI SD card module.
7. **SD board:** Small breakout module that holds the SD card and exposes it over SPI pins the ESP32 can talk to its adapter.
8. **Wires:** Wires connecting all the above to the ESP32's GPIO pins.

Process Step by Step

- Boot: mount SD card, connect to Wi-Fi, sync time via NTP, initialize the RFID reader, buzzer, and LCD.
- Main loop: continuously poll the RFID reader for a card.
- Card detected → read UID → look up the UID in the on-device student database (STUDENT_DB).
- Unknown UID: LCD shows “Unknown Card,” long buzz (3s). Nothing is logged; loop continues.
- Known UID: LCD shows the card holder's name; short beep (0.3s) confirms the card was accepted — this is the first of the two beeps.
- The board sends a POST request to the Flask server's /check face endpoint with the claimed student id
- The Flask server fetches a fresh snapshot from the ESP32-CAM and compares it against the reference photos on file.
- Result: match → short beep (second beep), “Correct” on the LCD, record saved to SD and Fire store.
- Result: false identity (a different known student's face matched better) → long buzz, “Wrong Identity” on the LCD, logged separately as a security alert — never written to the attendance collection.
- Result: No match / no face detected → Long Buzz, “Incorrect” on the LCD, nothing is logged.
- LCD resets to “Scan your card” and the loop continues.

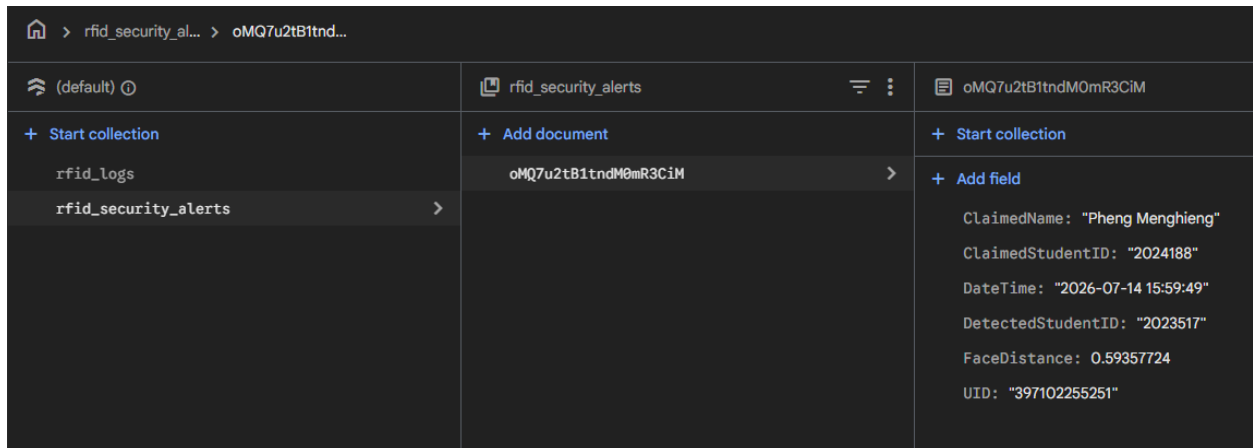
Thonny:

```
STUDENT_DB = {  
    "1425411918150": {  
        "name": "Khorn Sokhadom",  
        "student_id": "2023517",  
        "major": "Computer Science"  
    },  
    "397102255251": {  
        "name": "Pheng Menghieng",  
        "student_id": "2024188",  
        "major": "CS"  
    },  
}
```

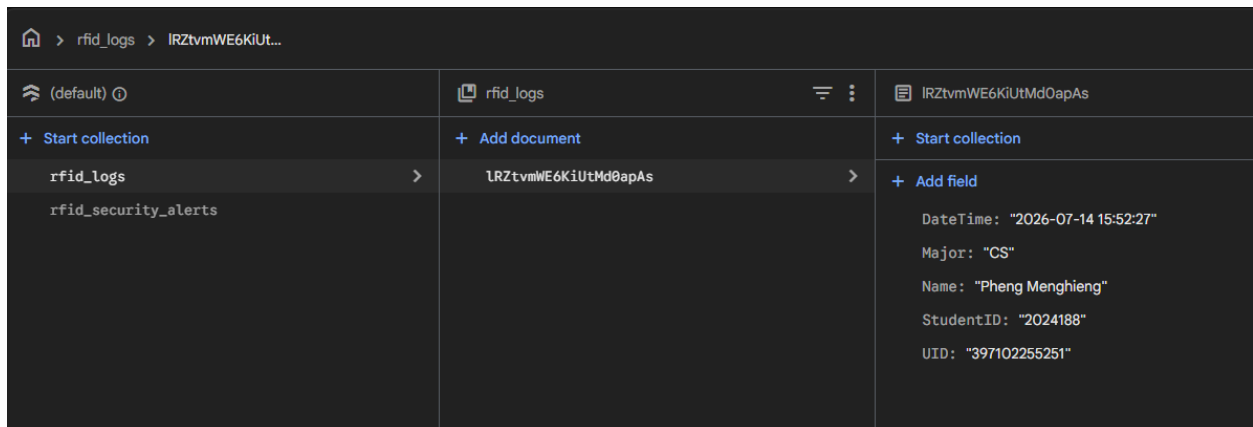
Reference Photos:

```
reference_faces/
  2024188/
    angle1.jpg
    angle2.jpg
    ... (3-8 photos)
  2023517/
    front.jpg
    ...
```

Unsuccessful attempt:



Successful attempt:



Stage	LCD Message	Buzzer
Card recognized	Card: Correct / <name>	Short beep (0.3s) — beep #1
Checking face	Checking face...	—
Face matched	Correct / <name>	Short beep (0.3s) — beep #2
Face mismatch	Incorrect / Face mismatch	Long buzz (3s)
False identity detected	Wrong Identity / Not <name>	Long buzz (3s)

Unknown card	Incorrect / Unknown Card	Long buzz (3s)
--------------	--------------------------	----------------

Resolved Issues

- Face check silently failing after a reference photo folder restructure caused by the server still expecting one photo per student_id.jpg instead of a per-student subfolder; fixed by rewriting the loader to walk subfolders and aggregate multiple encodings per student.
- False identity attempts (right card, wrong face) being logged as valid attendance caused by the original matcher only ever checking the live face against the claimed student's own photos, with no awareness of other known students; fixed by comparing against all known students and flagging cases where a different student is the closer match.

Limitations

- Fire store security rules are currently fully open (allow read, write: if true), suitable for a lab/demo environment but not for production use. Any client with the project ID can read or write data.
- ESP32-CAM snapshot quality is lower than a dedicated webcam, which can affect encoding precision, especially in poor lighting.
- Both the ESP32-CAM and the PC use DHCP-assigned local IPs, which are hardcoded into the other components config.
- Uses two devices to run, one for the web server and another for the card scan
- Using the same Wi-Fi network provides easy access to run the system, if not then changes on different IP or location must match perfectly or else it won't run or work properly.

References

GitHub for file code:

<https://github.com/PHENGMenghieng/iotproject/tree/main>